

DPO + layer streaming on a T4: DON'T SHIP (yet)

Qwen2.5-0.5B-Instruct, LoRA r16 on q/k/v/o, DPO beta 0.1, 400 pairs x 2 epochs (100 steps), `stream_layers: true`, soup-cli 0.75.1 on a Colab T4 (fp16, no bf16). Data: 500 clean pairs from `argilla/distilabel-intel-orca-dpo-pairs` (400 train, 100 held out). Every number below comes from `results/` (raw T4 logs) unless marked *rehearsal* (RTX 4060 Ti, `rehearsal/`).

Verdict: DON'T SHIP. The run is real: the saved adapter learned the preference and gets 85% on held-out pairs. But on the 29 held-out pairs where the better answer is the *longer* one, it gets 59% (17/29), which is not distinguishable from chance (one-sided $p = 0.23$). A length-balanced control run cut the length dependence by two-thirds and left that 59% unchanged. On top of that, fp16 silently skipped 4 optimiser steps, and none of Soup's checks (lint, pre-flight, ship) flagged either problem. What must change is in section 4.

1. Memory budget

Worked out from the model config before training (`memory_budget.py`), batch 2 x 512 tokens. DPO concatenates chosen and rejected, so a step is 4 rows = 2,048 tokens.

Term	GB
2 streamed layer buffers (fp16)	0.06
Tied embedding / LM head, resident	0.27
LoRA weights + grads + Adam (2.16M params x 16 B)	0.04
Checkpointed layer inputs + one layer's recompute	0.16
Policy logits + loss: 2,048 tok x 151,936 vocab x 14 B	4.36
First estimate	4.89
+ reference-pass logits alive at the same time (x 4 B)	1.25
Corrected estimate	6.14

Measured on the T4: peak allocated 5.6 GiB = **6.01 GB** (Soup's own `gpu_mem`), and `nvidia-smi` peaked at **9.85 GB**. Soup's pre-flight predicted 4.88 GB.

The gap, explained. Logits dominate: the vocab is 152k, so 90% of the peak is one tensor shape. My first estimate, and Soup's, treat the DPO reference pass as transient. It isn't. In TRL 0.29 `_compute_loss` runs the reference forward *after* the policy forward, while the policy's logits and graph are still alive, so the reference logits and their `.contiguous()` shifted copy (2 + 2 B per element) stack on top. Two rehearsal points, batch 1 (3.22 GB) and batch 2 (6.01 GB), give a slope of **17.9 B per logit element** against the 14 assumed, and the corrected estimate lands within 2% of both. The real peak is 23% above Soup's pre-flight number. A pre-flight that under-predicts can pass a config that then runs out of memory (or, per Soup's own comments, spills to host memory silently on Windows).

`nvidia-smi` is higher again: it also counts the CUDA context and memory the allocator reserved but isn't using. Soup tried to enable `expandable_segments` to reduce that, and its own log says it couldn't because CUDA was already initialised. The pinned host copy of the base (0.72 GB of RAM) appears in no VRAM number.

2. Proving it trained

Falling loss doesn't prove learning. It comes from the policy in memory, not the adapter written to disk; DPO loss also falls when the model pushes *both* answers down, the rejected one faster; and it falls on pairs the model is memorising. So `verify_training.py` loads the **saved adapter from disk** into a plain resident PEFT model, independent of Soup's streamed trainer, and checks:

Check	Result (T4)
Every target (24 layers x 4 modules) has non-zero <code>lora_B</code>	96/96, $ dW / W $ 0.08%-0.57%, even across depth
Saved <code>ref/</code> adapter is the identity (all-zero B)	yes
Train pairs: reward-margin accuracy / mean margin (95% CI)	85% , 1.99 (1.59-2.42)
Held-out pairs, never trained on	85% , 1.74 (1.29-2.19), sign test $p = 5e-13$
Control: same adapter, B zeroed	margin exactly 0
Control: random B, same per-tensor norms	held-out 45%, margin -0.008

The random-direction control matters most: it moves the weights by the same amount and scores at chance, so the gain comes from the direction the model learned.

What this check does not detect: whether the labels are right (it measures agreement with the data, noise included), generation quality, behaviour on Russian support tickets, and *why* the model prefers what it prefers. That last gap matters here (next section).

3. Silent failures

What could go wrong	How I checked	Does Soup catch it?	Evidence
Length shortcut. DPO sums log-probs over tokens; chosen is shorter in 65% of pairs	Held-out accuracy split by which answer is longer, plus a length-balanced control run	No. <code>data lint</code> says MINOR on raw data, OK on the cleaned set ($d = -0.27$)	Held-out 95% when chosen is shorter, 59% when longer (17/29). Length-balanced control: 83% / 59% , correlation -0.36 to -0.13
fp16 skipped steps. The T4 has no bf16, so the GradScaler skips any step with inf grads	<code>grad_norm</code> in <code>trainer_state.json</code>	No. The loss watchdog is off by default, and it only watches the loss, which stays finite when a step is skipped	4 of 100 steps skipped. LR sat at 0 for steps 1-3 because skipped steps don't advance the scheduler
Ties in the data (no preference signal)	Dataset <code>status</code> field	No. <code>identical_pairs</code> compares strings; ties aren't identical	168 of 500 raw rows (34%) are ties; I dropped them
A lint check that never ran	Read the check	No. It reports OK	<code>near_duplicates</code> : OK - "skipped (datasketch not installed)": [train] doesn't install it
Truncation at <code>max_length 512</code> (prompt capped at 256)	Re-tokenised every pair	No check for DPO	Prompt cut in 19.5% of pairs; both answers lose their end-of-turn in 5.5%
DPO metrics thrown away	Queried Soup's run DB	n/a	<code>metrics</code> table has loss/lr/grad_norm only. Rewards, margins, accuracies exist only in checkpoint JSON
soup ship can't see any of this	Ran it on the trained adapter, a zero adapter and a random adapter	No	All three got DON'T SHIP with the same reason (<code>task_win</code>). On the task eval, zero and random scored exactly base (8/18) and the real adapter scored <i>worse</i> (6/18). Ship also loaded bf16 on the T4
Russian data costs more tokens	Same sentences, EN vs RU	No	1.52x the tokens, so truncation will be worse on the real package

4. Verdict and what must change

DON'T SHIP. The pipeline works and the adapter really learned, but the evidence says it learned *brevity* more than *quality*: where length can't help, it's at chance. Before shipping:

1. **Get a real quality signal first.** Balancing lengths reduced the shortcut but didn't raise the 59%, so the fix is more and better pairs (not ties, not length-driven), or a length-normalised objective. The gate: held-out accuracy on chosen-longer pairs, with a sample big enough to beat chance.
2. **Gate on skipped fp16 steps**, or train on a bf16 card. Here 4 of 100 steps did nothing and no log line said so.
3. **Evaluate on real tickets.** Held-out pairs from the same distribution say nothing about Russian support data.
4. **Change Soup so this can't pass ship again** (patches in `soup-patches/`, tests included, each fails without its fix):
 - `soup ship` refuses a no-op adapter (all `lora_B` zero) before loading anything. A random adapter still gets through, so the next step is a reward-margin gate: run `verify_training.py`'s held-out margin check, with the random-direction control, inside `ship` for DPO runs.
 - `data lint` reports a skipped check as **SKIPPED**, not OK.
 - `ship` loads fp16 on cards without bf16 (it hard-coded bf16, so on a T4 it evaluated in a different precision from training).
 - Proposed, not built: add the reference-pass term to the DPO VRAM estimate, and store DPO reward metrics in the run DB.

What surprised me and what still concerns me: [SURPRISES.md](#). How I used AI: [AI USAGE.md](#).